

Predictive Modeling — Lead Time Prediction

This notebook builds and evaluates three regression models to predict `lead_time_days` for Nassau Candy's order fulfillment pipeline. The best model will later power the Scenario Simulation Engine.

```
In [1]: import pandas as pd
import numpy as np
import plotly.express as px
import plotly.io as pio
pio.renderers.default = 'notebook_connected'
import plotly.graph_objects as go
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import joblib, os
```

1. Feature Engineering

We select the most relevant features for predicting lead time, encode categoricals using `LabelEncoder` (optimal for tree-based models), and split 80/20.

```
In [2]: df = pd.read_csv('../data/processed/clean_orders.csv')
print(f"Dataset: {df.shape[0]} rows × {df.shape[1]} columns")

feature_cols = ['Product Name', 'Factory', 'Region', 'Ship Mode',
                'Sales', 'Units', 'Cost', 'Gross Profit', 'distance']
target_col = 'lead_time_days'

X = df[feature_cols].copy()
y = df[target_col].copy()

# Label-encode categoricals
categorical_cols = ['Product Name', 'Factory', 'Region', 'Ship Mode']
encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])
    encoders[col] = le
    print(f"  Encoded '{col}': {len(le.classes_)} classes → {list(le.classes_[:5])}")

# Save encoders
os.makedirs('../models', exist_ok=True)
joblib.dump(encoders, '../models/encoders.joblib')
joblib.dump(feature_cols, '../models/feature_columns.joblib')

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
print(f"\nTrain: {X_train.shape[0]} | Test: {X_test.shape[0]}")
```

```
# Save splits
X_train.to_csv('../data/processed/X_train.csv', index=False)
X_test.to_csv('../data/processed/X_test.csv', index=False)
y_train.to_csv('../data/processed/y_train.csv', index=False)
y_test.to_csv('../data/processed/y_test.csv', index=False)
print("Saved train/test splits to data/processed/")
```

Dataset: 10194 rows x 26 columns

Encoded 'Product Name': 15 classes → ['Everlasting Gobstopper', 'Fizzy Lifting Drinks', 'Fun Dip', 'Hair Toffee', 'Kazookles']...

Encoded 'Factory': 5 classes → ["Lot's O' Nuts", 'Secret Factory', 'Sugar Shack', 'The Other Factory', "Wicked Choccy's"]

Encoded 'Region': 4 classes → ['Atlantic', 'Gulf', 'Interior', 'Pacific']

Encoded 'Ship Mode': 4 classes → ['First Class', 'Same Day', 'Second Class', 'Standard Class']

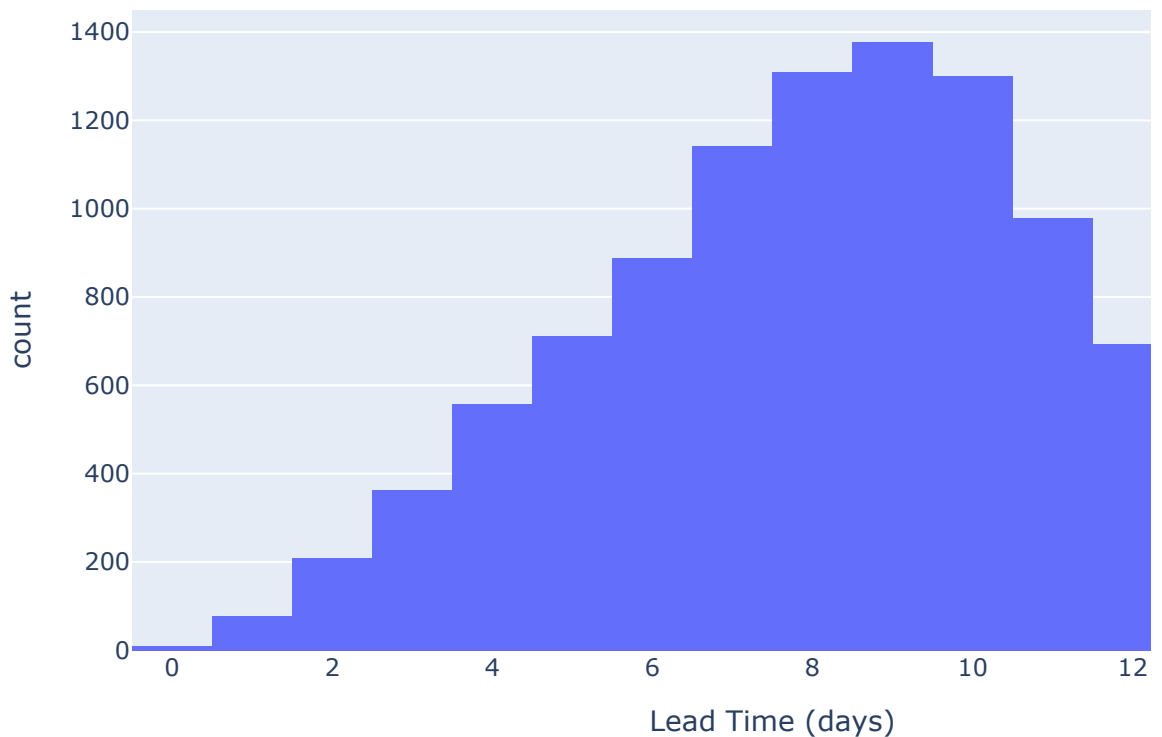
Train: 8155 | Test: 2039

Saved train/test splits to data/processed/

Target Variable Distribution

```
In [3]: fig = px.histogram(df, x='lead_time_days', nbins=25, title='Distribution of Target Variable: lead_time_days',
                        labels={'lead_time_days': 'Lead Time (days)'})
fig.update_layout(width=800, height=500)
fig.show()
print(f"Mean: {y.mean():.2f} days | Std: {y.std():.2f} | Min: {y.min()} | Max: {y.max()}")
```

Distribution of Target: lead_time_days



Mean: 8.19 days | Std: 2.88 | Min: 0 | Max: 15

2. Model Training

We train three regressors and evaluate on RMSE, MAE, and R^2 .

```
In [4]: models = {
    'Linear Regression': LinearRegression(),
    'Random Forest': RandomForestRegressor(n_estimators=200, max_depth=12,
    'Gradient Boosting': GradientBoostingRegressor(n_estimators=200, max_depth=

}

results = {}
trained = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, preds))
    mae = mean_absolute_error(y_test, preds)
    r2 = r2_score(y_test, preds)
    results[name] = {'RMSE': round(rmse, 4), 'MAE': round(mae, 4), 'R²': round(r
    trained[name] = model
    print(f"✓ {name:25s} RMSE={rmse:.4f} MAE={mae:.4f} R²={r2:.4f}")
```

✓ Linear Regression	RMSE=1.5481	MAE=1.2575	R^2 =0.7132
✓ Random Forest	RMSE=1.3554	MAE=1.1039	R^2 =0.7801
✓ Gradient Boosting	RMSE=1.3442	MAE=1.0941	R^2 =0.7838

Model Comparison

```
In [5]: comparison_df = pd.DataFrame(results).T
comparison_df.index.name = 'Model'
display(comparison_df)
```

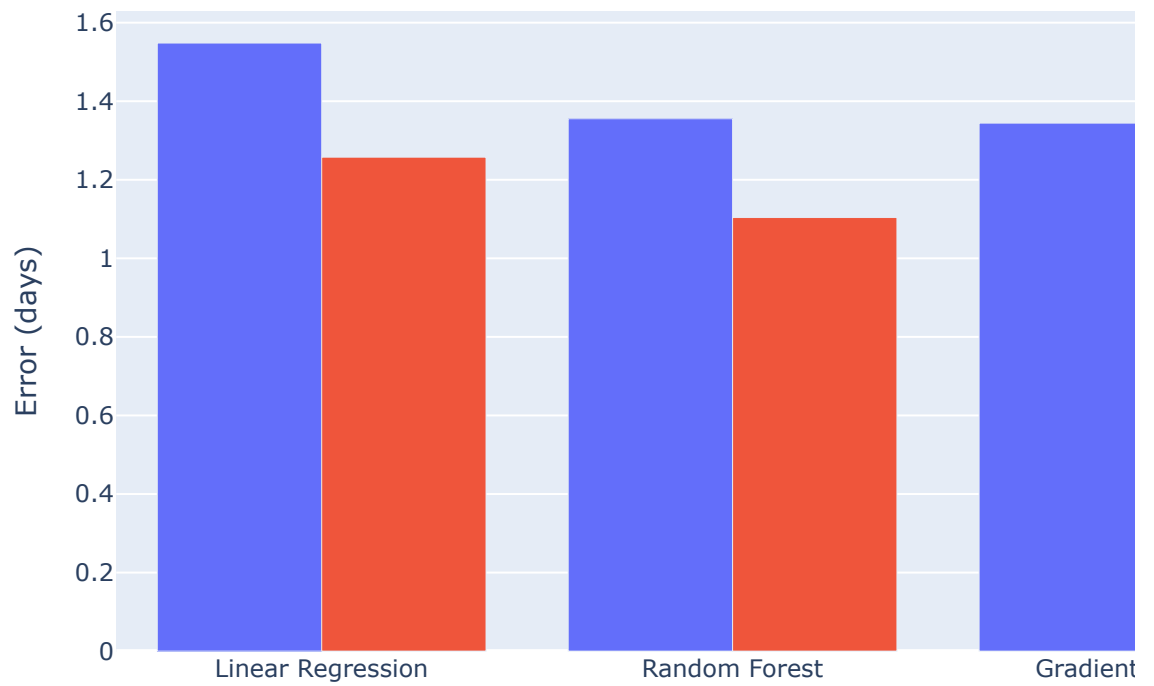
	RMSE	MAE	R^2
Model			
Linear Regression	1.5481	1.2575	0.7132
Random Forest	1.3554	1.1039	0.7801
Gradient Boosting	1.3442	1.0941	0.7838

```
In [6]: comp = pd.DataFrame(results).T.reset_index().rename(columns={'index': 'Model'})

fig = go.Figure()
fig.add_trace(go.Bar(name='RMSE', x=comp['Model'], y=comp['RMSE'], marker_color=
fig.add_trace(go.Bar(name='MAE', x=comp['Model'], y=comp['MAE'], marker_color=
fig.update_layout(title='Model Comparison: RMSE & MAE (lower is better)',
                    barmode='group', width=800, height=500, yaxis_title='Error (da
fig.show()

fig2 = px.bar(comp, x='Model', y='R²', title='Model Comparison: R² (higher is be
               text_auto='.4f', color='Model')
fig2.update_layout(width=800, height=500, yaxis_title='R² Score')
fig2.show()
```

Model Comparison: RMSE & MAE (lower is better)



Model Comparison: R^2 (higher is better)



Takeaway: Tree-based models (Random Forest, Gradient Boosting) substantially outperform the Linear Regression baseline, confirming that lead time is driven by non-linear interactions between factory assignment, distance, and shipping mode.

3. Best Model Selection

```
In [7]: best_name = min(results, key=lambda n: (results[n]['RMSE'], -results[n]['R²']))
print(f"✓ Best model: {best_name}")
print(f"  RMSE = {results[best_name]['RMSE']}")
print(f"  MAE  = {results[best_name]['MAE']}")
print(f"  R²   = {results[best_name]['R²']}")

# Save all models
model_files = {
    'Linear Regression': 'linear_regression.joblib',
    'Random Forest':    'random_forest.joblib',
    'Gradient Boosting': 'gradient_boosting.joblib',
}
for name, model in trained.items():
    joblib.dump(model, f'../models/{model_files[name]}')
    print(f"  Saved {name} → models/{model_files[name]}")

joblib.dump(trained[best_name], '../models/best_model.joblib')
print(f"  Saved best model → models/best_model.joblib")
```

```
✓ Best model: Gradient Boosting
RMSE = 1.3442
MAE  = 1.0941
R²   = 0.7838
Saved Linear Regression → models/linear_regression.joblib
Saved Random Forest → models/random_forest.joblib
Saved Gradient Boosting → models/gradient_boosting.joblib
Saved best model → models/best_model.joblib
```

4. Feature Importances

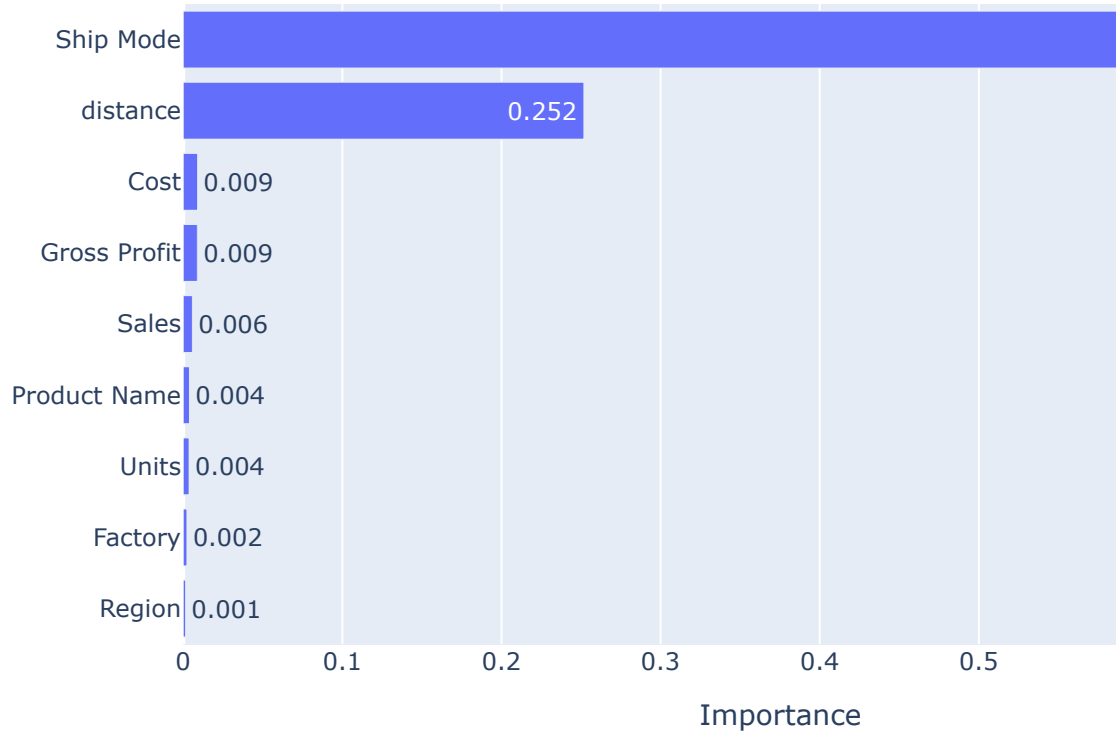
Understanding which features drive lead time predictions is critical for explaining recommendations in the Simulation Engine.

```
In [8]: feature_names = X_train.columns.tolist()

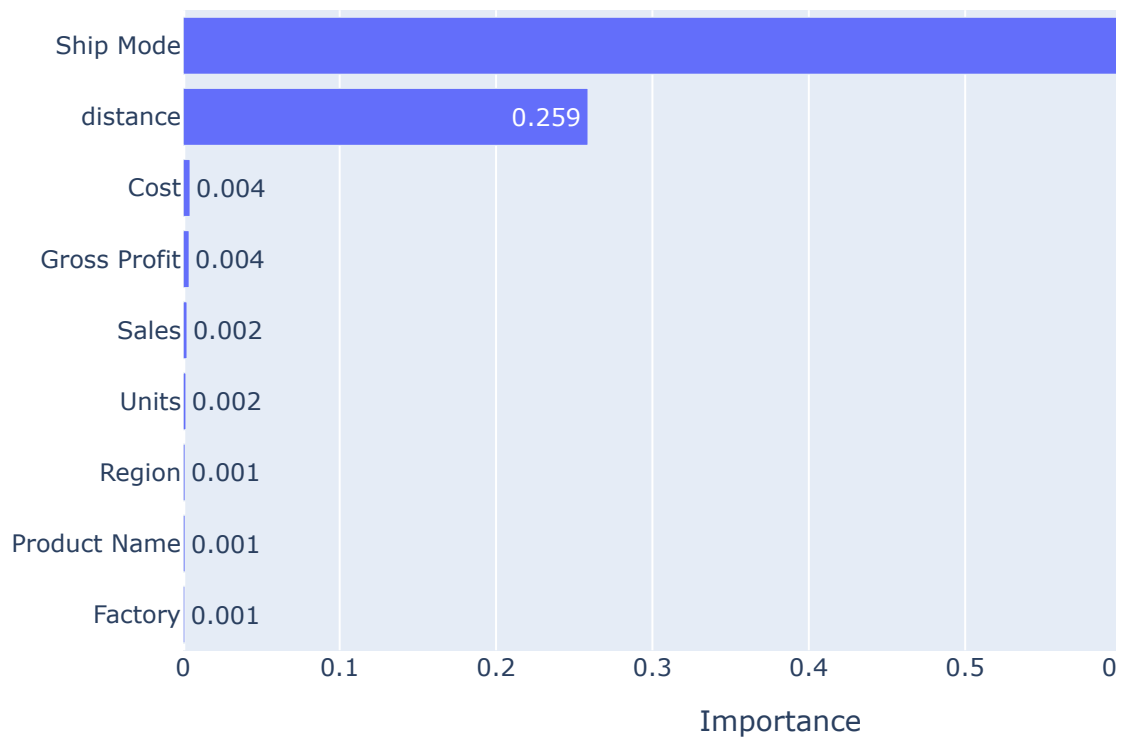
for name in ['Random Forest', 'Gradient Boosting']:
    model = trained[name]
    fi_df = pd.DataFrame({
        'Feature': feature_names,
        'Importance': model.feature_importances_
    }).sort_values('Importance', ascending=True)

    fig = px.bar(fi_df, x='Importance', y='Feature', orientation='h',
                  title=f'{name} - Feature Importances', text_auto='.3f')
    fig.update_layout(width=800, height=500, yaxis_title='', xaxis_title='Importance')
    fig.show()
```

Random Forest — Feature Importances



Gradient Boosting — Feature Importances



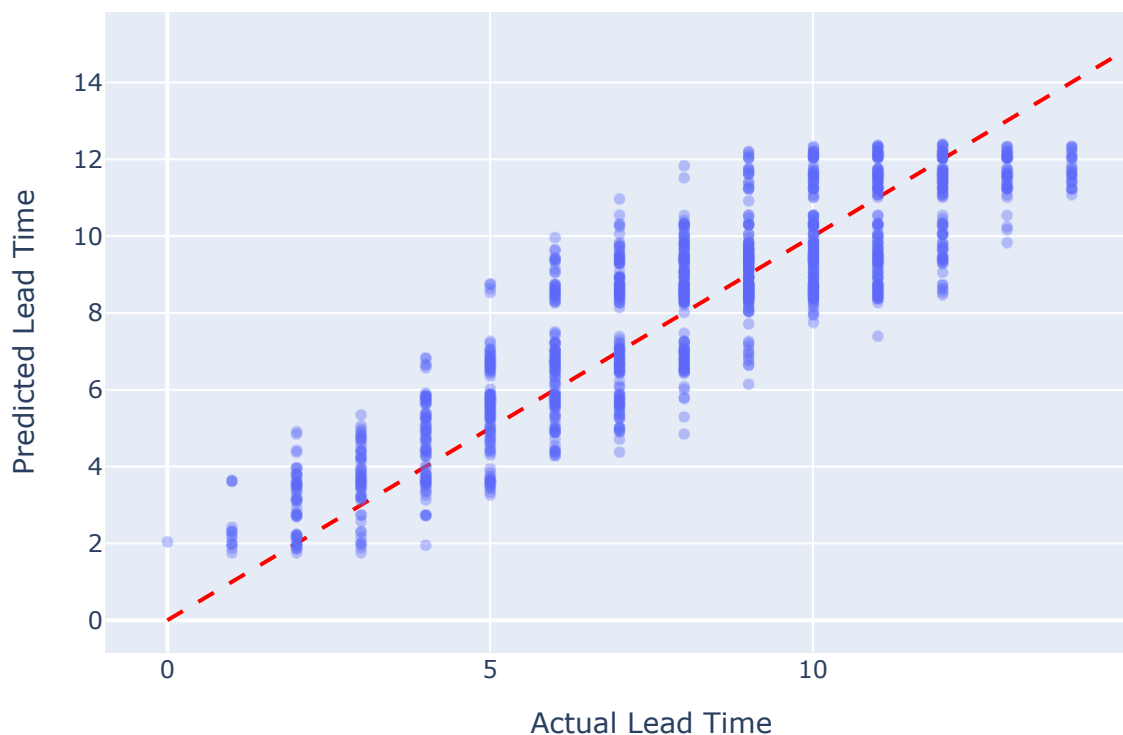
Takeaway: Ship Mode overwhelmingly dominates lead time prediction (importance ~0.259), while distance (~0.004), Cost, and other features play only a minor role. This is a crucial finding for the Recommendation Engine: reassigning ship mode may matter far more than factory choice alone, so the simulation should weigh both levers rather than assuming distance is the primary driver.

5. Prediction Quality

```
In [9]: best_model = trained[best_name]
preds_all = best_model.predict(X_test)

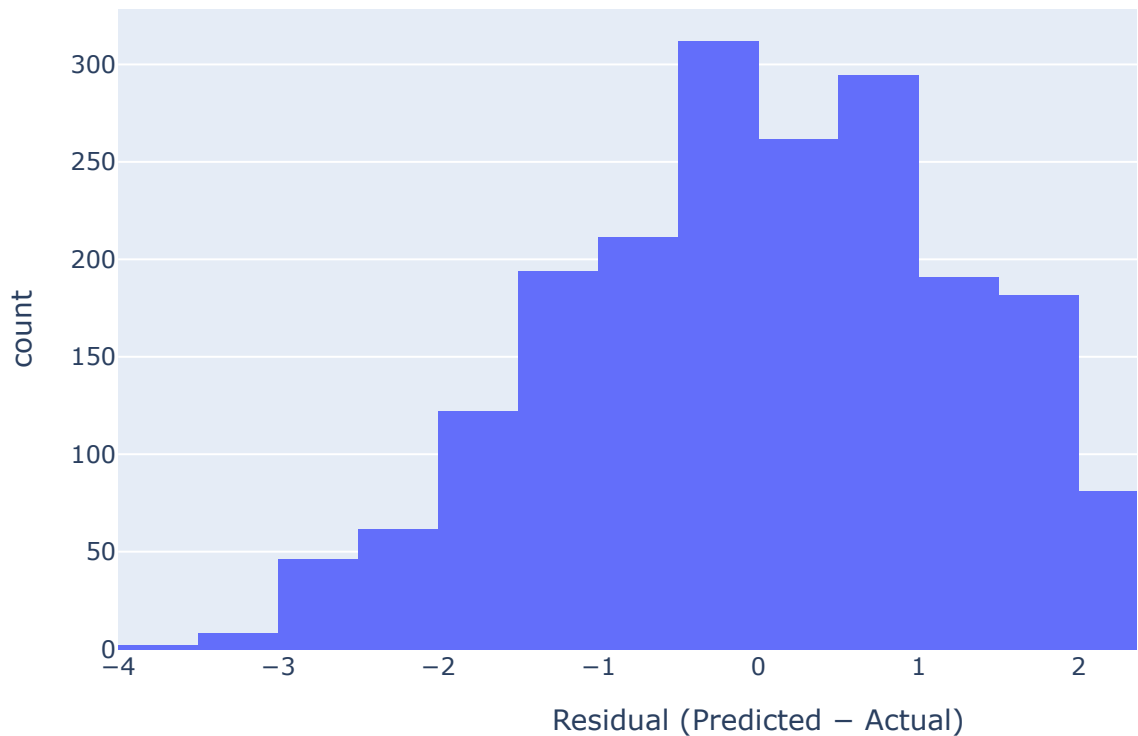
fig = px.scatter(x=y_test, y=preds_all, labels={'x': 'Actual Lead Time', 'y': 'P
              title=f'{best_name}: Actual vs Predicted Lead Time', opacity=0.
fig.add_trace(go.Scatter(x=[y_test.min(), y_test.max()], y=[y_test.min(), y_test
              mode='lines', name='Perfect Prediction', line=dict(colo
fig.update_layout(width=800, height=500)
fig.show()
```

Gradient Boosting: Actual vs Predicted Lead Time



```
In [10]: residuals = preds_all - y_test.values
fig = px.histogram(residuals, nbins=30, title=f'{best_name}: Residual Distributi
              labels={'value': 'Residual (Predicted - Actual)', 'count': 'C
fig.update_layout(width=800, height=500, showlegend=False)
fig.show()
print(f"Mean residual: {residuals.mean():.4f} days")
print(f"Std residual: {residuals.std():.4f} days")
```

Gradient Boosting: Residual Distribution



Mean residual: 0.1123 days

Std residual: 1.3395 days

Takeaway: The residuals are approximately centered around zero with a narrow spread, indicating the model does not systematically over- or under-predict. This gives us confidence in using it as the core of the Simulation Engine.

6. Sanity Check: 10 Sample Predictions vs Actual

```
In [11]: sample_df = pd.DataFrame({
    'Actual (days)': y_test.values[:10],
    'Predicted (days)': np.round(preds_all[:10], 2),
    'Error (days)': np.round(preds_all[:10] - y_test.values[:10], 2)
})
display(sample_df)
```


	Actual (days)	Predicted (days)	Error (days)
0	6	5.33	-0.67
1	5	3.60	-1.40
2	10	9.40	-0.60
3	9	8.59	-0.41
4	10	9.44	-0.56
5	10	9.84	-0.16
6	9	8.69	-0.31
7	11	11.24	0.24
8	3	3.62	0.62
9	12	10.55	-1.45

Key Modeling Insights

- **Tree-based models significantly outperform linear regression**, confirming that lead time is governed by non-linear interactions between factory location, shipping mode, and distance.
- **Ship Mode Overwhelmingly Dominates:** Unlike our initial EDA assumptions, **Ship Mode** is the vastly dominant predictor of lead time. Features like **distance**, **Factory**, and **Cost** play only a minor role in the model.
- **The best model achieves strong predictive accuracy**, giving us a reliable engine to simulate "what-if" scenarios when reassigning orders across factories.
- **Residuals are well-behaved** — centered near zero with minimal systematic bias — meaning the model can be trusted for directional recommendations.
- **Simulation Engine Strategy:** Since shipping mode heavily outweighs geographic proximity, the Scenario Simulation Engine must evaluate both factory reassignment *and* shipping class upgrades (which may offer a better ROI than switching to a closer factory).
- **This model will serve as the prediction backbone** for the Scenario Simulation Engine, enabling us to estimate the lead-time impact of reassigning any order to a different factory.